
传送门作用下的二维泊松方程有限元求解

摘要：

本文讨论了简单传送门作用下的二维泊松方程有限元求解方法。泊松方程广泛出现在工程和物理领域，实际应用中，三维曲面往往具有不同于平面的拓扑连通性。本文中传送门为一种线段形的空间切割、粘合边界，能够改变平面求解域的连通性，提供了一种在平面上求解圆柱面、环面、球面等问题的简单方法。针对上述问题，本文基于 Galerkin 变分原理推导了 P1 空间拉格朗日线性有限元弱形式，使用特殊函数定义了传送门端点网格加密方式，依托 gmsh 工具完成了网格划分以及对网格单元连通性的重构，实现了传送门作用下的齐次 Dirichlet 边界条件泊松方程求解。此外，本文进一步讨论了求解方法的稳定性和收敛性、传送门的简单实际应用。

关键词： 有限元法；二维泊松方程；传送门；拓扑连通性；Galerkin 变分原理

1 引言

泊松方程是经典的二阶椭圆型偏微分方程，广泛应用于物理学、工程力学、电磁场分析、热传导模拟、流体力学等众多领域。在实际场景中，大量物理场稳态分布问题均可转化为泊松方程求解问题，如曲面稳态温度场、静电势分布、流体压力场分布等，因此精准、高效地实现泊松方程的数值求解具有重要实际价值。

有限元方法是偏微分方程数值求解的一大主流，其鲁棒性强、求解精度高、稳定性好，是求解泊松方程的不二选择。一般有限元求解泊松方程，仅限于平面单连通、多连通常规求解域。在实际工程场景中，很多问题的求解域不是平面，而是圆

柱面、环面、球面及更复杂的三维曲面，这类曲面具备特殊的空间拓扑性质，与二维平面存在本质差异，无法直接通过一般有限元网格完成求解。

将三维曲面展开到二维平面进行处理可以极大减少计算量。目前主流的展开方式，其一是在分割切口上构建特殊等效边界条件，其二是在数值求解迭代过程中，对分割边界节点建立使用映射表关联。这两种方法相对成熟，但前者引入了非齐次边界条件，使得方程变得更加复杂，可能影响求解精度或迭代收敛效率，是一种非本质的方法；后者需要使用能处理节点映射关系的定制化的求解器，无法兼容很多通用的有限元求解算法。本文通过直接修改二维网格的单元连接关系，实现三维曲面拓扑结构的等效还原，避免了额外边界条件，无需依赖定制求解器，同时一定程度上兼顾数值稳定性。

基于以上思路，本文使用经典 Galerkin 变分原理，推导了齐次 Dirichlet 边界条件下泊松方程的 P1 线性有限元弱形式与离散方程组，结合 gmsh 工具完成含传送门拓扑结构的自适应网格剖分与网格连通性修改，实现了拓扑曲面泊松方程的二维数值求解。本文通过分析解的图像、对比数值解与解析解，验证了方法的稳定性与收敛性。后将该方法应用于圆柱面、平直环面、克莱因瓶、球面等三维曲面的稳态场数值模拟，证实了修改网格结构的可行性，为复杂拓扑曲面的偏微分方程数值求解提供了一种简洁可靠、普适性强的思路。

2 正文

本文主要讨论了二维平面上，传送门作用下，含齐次 Dirichlet 边界条件的泊松方程的求解。

方程形如

$$-\Delta w = f \text{ in } \Omega, \quad w = 0 \text{ on } \partial\Omega \quad 2.1$$

其中 Ω 为方程的求解域，是一个二维区域或三维曲面； w 为待求函数； f 为任意 V 中的函数， V 定义为

$$V = H_0^1(\Omega) = \{v: v \text{ 在 } \Omega \text{ 内连续, 在 } \partial\Omega \text{ 上为 } 0\}$$

本文使用了修改节点连接的方式实现传送门模拟，无需引入新的边界条件。

2.1 有限元离散形式推导

本文讨论的问题的有限元离散形式推导可分为四个部分。

2.1.1 弱形式的推导

根据实变函数理论，解方程 2.1 等价于求 w ，使得对任意试函数 v ，以下等式恒成立

$$\int_{\Omega} -v \nabla w \, d\Omega = \int_{\Omega} f v \, d\Omega$$

使用分部积分及格林公式，等式左端可化为

$$-\int_{\Omega} v \Delta w \, d\Omega = -\int_{\partial\Omega} v \frac{\partial w}{\partial n} \, ds + \int_{\Omega} \nabla v \cdot \nabla w \, d\Omega = \int_{\Omega} \nabla v \cdot \nabla w \, d\Omega$$

最终方程 2.1 转化为，求 $w \in V$ ，使得对任意 $v \in V$ 下式恒成立

$$\int_{\Omega} \nabla v \cdot \nabla w \, d\Omega = \int_{\Omega} f v \, d\Omega \quad 2.2$$

式 2.2 称为 2.1 的**弱形式**。式 2.2 仅使用梯度和积分表示，能够在 P1 空间（分段线性函数空间）上近似求解。

2.1.2 网格划分的定义

为了对 w 和 v 进行 P1 空间近似，现用三角剖分 $\mathcal{T}_h$ 近似覆盖 Ω ，其中 h 表示网格大小。记所有节点为

$$\mathcal{P} = \{p_i: p_i = (x_i, y_i), i = 1, \dots, N\}$$

所有三角形可以表示为三元数组组成的集合

$$\mathcal{K} = \{(i_1, i_2, i_3): p_{i_1}, p_{i_2}, p_{i_3} \text{ 组成 } \mathcal{T}_h \text{ 中三角形}\}$$

边界点可表示为

$$\mathcal{B} = \{i: p_i \text{ 在 } \mathcal{T}_h \text{ 边界上}\}$$

2.1.3 P1 空间及基函数的定义

考虑如下的分段线性函数空间

$$V_h = \{v_h \in C^0(\Omega): v_h \text{ 在 } \mathcal{K} \text{ 中三角形上为线性函数}\}$$

在 V_h 上取基函数 ϕ_i 满足

$$\phi_i(p_j) = \begin{cases} 1, & j = i \\ 0, & j \neq i \end{cases}$$

$w(x, y)$ 存在 V_h 上的近似函数, 仍记为 $w(x, y)$, 则

$$w(x, y) = \sum_{i=1}^N W_i \phi_i(x, y), \quad \text{且 } \forall i \in \mathcal{B} \text{ 有 } W_i = 0$$

对基函数 $\phi_i(x, y)$, 其在相邻三角形 $K \in \mathcal{K}$ 上的梯度为

$$\nabla \phi_i^K = -\frac{1}{2|K|} \begin{bmatrix} y_2 - y_3 \\ x_3 - x_2 \end{bmatrix}, \quad x_{2,3}, y_{2,3} \text{ 为另外两点坐标, } |K| \text{ 为三角形面积}$$

2.1.4 方程离散形式的推导

对方程 2.2 中 w , v 取 P1 空间近似, 原方程可使用基函数表示为

$$\int_K \sum_{i=1}^N W_i \nabla \phi_i(x, y) \cdot \sum_{i=1}^N V_i \nabla \phi_i(x, y) dx = \int_K f \sum_{i=1}^N V_i \phi_i(x, y) dx, \quad \forall K \in \mathcal{K}$$

每个三角形上, 只有顶点对应的三个基函数非 0, 于是方程进一步写为

$$\int_K \sum_{i=1}^3 W_{K[i]} \nabla \phi_{K[i]}(x, y) \cdot \sum_{j=1}^3 V_{K[j]} \nabla \phi_{K[j]}(x, y) dx = \int_K f \sum_{i=1}^3 V_{K[i]} \phi_{K[i]}(x, y) dx$$

由试函数 $V_i (i = 1, 2, \dots, N)$ 的任意性, 这等价于

$$\left(\int_K \nabla \phi_{K[i]} \cdot \nabla \phi_{K[j]} dx \right)_{i \times j} \begin{bmatrix} W_{K[1]} \\ W_{K[2]} \\ W_{K[3]} \end{bmatrix} = \begin{bmatrix} \int_K f \phi_{K[1]}(x, y) dx \\ \int_K f \phi_{K[2]}(x, y) dx \\ \int_K f \phi_{K[3]}(x, y) dx \end{bmatrix}$$

上式可记为线性方程组 $A_K W_K = F_K, \forall K \in \mathcal{K}$ 。对每个 $W_i (i = 1, 2, \dots, N)$, 累加所有包含 W_i 的方程, 最终得到方程组

$$AW = F, \text{ 且 } W_i = 0 (i \in \text{边界 } \mathcal{B}) \quad 2.3$$

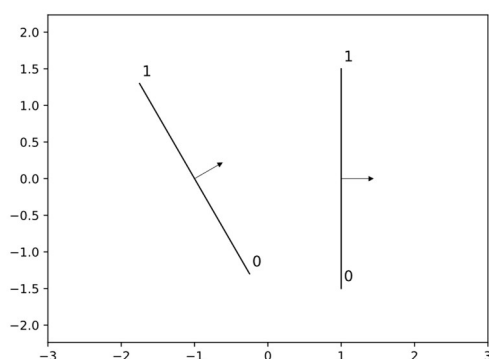
求解该方程组, 进而可通过 P1 基函数计算解 $w(x, y)$ 。

2.2 传送门及其在网格中的表达

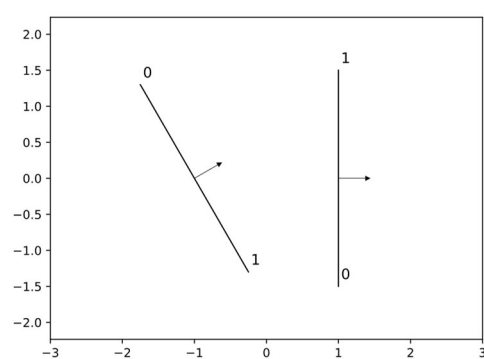
2.2.1 传送门的定义

传送门由一对线段构成，并区分正面与反面。每条线段由中点坐标、长度、正面方向决定。场或物体从任意一条线段的反面进入，将配对线段的正面穿出；从正面进入，则从配对线段的反面穿出。

每组传送门存在两种几何状态，未翻转与单门中心镜像翻转。未翻转状态下，两段线段可通过旋转平移重合对应点；镜像翻转状态下，其中一条线段沿自身法线镜像翻转，节点配对顺序倒置，无法通过平移旋转重合对应点。如图 2-1 所示的两对传送门，编号相同的端点为对应点。图(a)中传送门未翻转，平移、旋转后对应点可重合；图(b)中传送门存在翻转，平移、旋转后对应点无法重合。



(a) 一对未翻转传送门的示意图



(b) 一对翻转传送门的示意图

图 2-1 传送门翻转状态示意图

2.2.2 传送门在网格中的嵌入

传送门端点为空间上的“尖锐点”，在嵌入网格时，需考虑加密。下考虑一对传送门的嵌入。

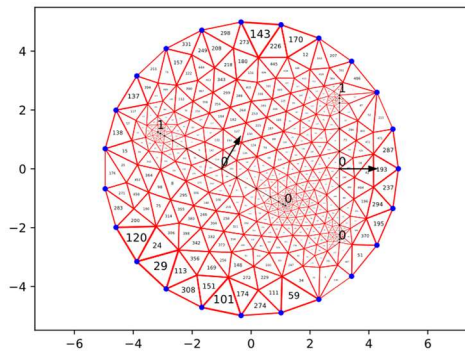
记传送门长度为 L ，为嵌入网格，考虑将两线段切割成对称等分布的小段，两端分布更密集。记其中最长段、最短段长度分别 $m = 0.02L$ 、 $M = 0.2L$ 。定义 $[-1,1]$ 上的余弦分布函数

$$f(t) = \cos^2\left(\frac{\pi}{2}x\right)(M - m) + m$$

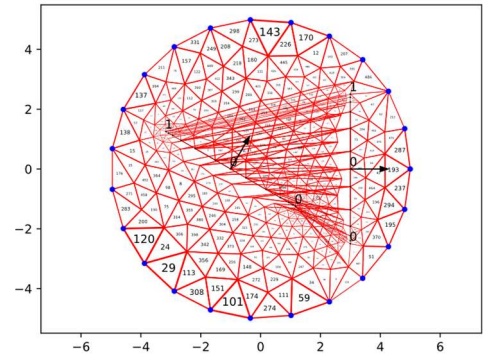
则 $f(-1) = f(1) = m$, $f(0) = M$, 在 $[-1,1]$ 区间上插入 $2n$ 个等分点, 则存在最小的 n 使得

$$\sum_{i=1}^n f\left(\frac{i}{n+1}\right) \geq L$$

使用 n 分割传送门, 得到每段长度分别为 $f\left(\frac{i}{n+1}\right)$, $i = 1, 2, \dots, 2n$ 分割方式。将传送门上的分割点直接定义为网格节点, 并保证每段小段上只有一个三角形单元, 最终生成全部格点。图 2-2 (a)为传送门嵌入后的网格分布示意图。



(a) 传送门嵌入示意图



(b) 节点重新映射后的网格示意图

图 2-2 传送门嵌入网格及节点重新映射方式示意图

2.2.3 传送门节点映射

网格生成后, 对个传送门, 找到其反面的所有临近三角形单元, 修改其在传送门上顶点, 映射到配对传送门的对应位置, 由此实现等效的空间切割与拼接。如图 2-2 (b)所示, 传送门反面所有三角形在门上的节点被重新映射到配对的门上, 在后续计算中, 场将沿着网格跨越平面空间, 同时避免了引入新的边界条件。

2.3 部分模拟结果展示

这里展示了一些边界形状和场源条件下的模拟结果。

2.3.1 圆形边界、中心场源、三对传门模拟结果

图 2-3 中, 使用了半径为 5 的圆形边界, 网格尺寸为 0.05 至 0.1, 中心放置半

径为 0.3, $f = 100$ 的场源。任意放置三对传送门, 传送门成功实现了传送效果, 可以将中心区域的场传送到外围, 且传送前后场强连续变化, 初步验证了方法的有效性。

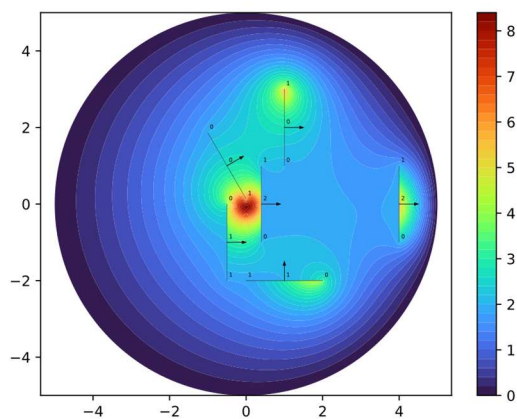


图 2-3 圆形边界、中心场源、三对传门模拟结果示意图

2.3.2 圆形边界、中心场源、一对平行传送门

图 2-4 展示了一种特殊情况, 此时边界、场源、传送门完全中心对称, 根据对称性, 传送门产生的影响被对称性抵消, 此时的解完全等于传送门不存在时的情形。

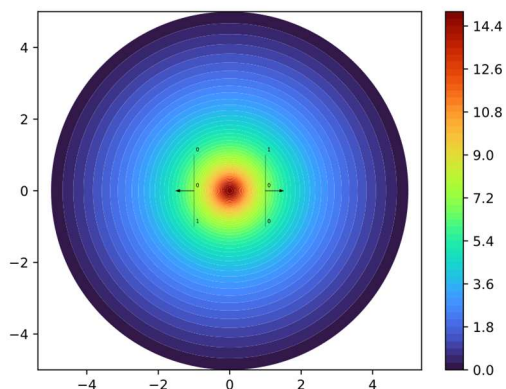


图 2-4 一对中心对称的平行传送门影响下解的图像

2.3.3 方形边界、两个条形场源

图 2-5 展示了另一种特殊情形下场的分布。使用方形边界，设置一对形状相等、大小相反的条形场源，在中间区域放置一对传送门。在没有传送门的情况下，中心区域的场近似为匀强场，可以模拟匀强重力环境。传送门的影响如图所示，上方传送门将高势能红色场传送到了下方。

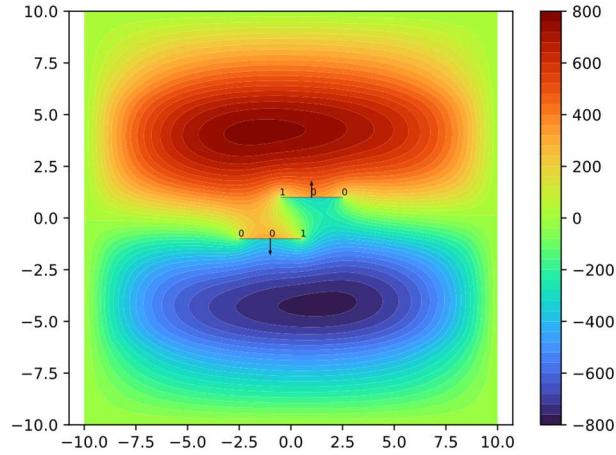


图 2-5 平行条形场源受传送门影响解的图像

2.4 解的稳定性及误差分析

包含传送门的问题无法计算解析解，故使用无门问题验证方法的数值稳定性与收敛性。考虑 $[-1,1]^2$ 上的函数

$$w(x, y) = \sin\left(\frac{\pi}{2}(x+1)\right) \sin\left(\frac{\pi}{2}(y+1)\right)$$

其满足齐次 Dirichlet 边界条件，带入泊松方程计算得源项

$$f(x, y) = \frac{\pi^2}{2} \sin\left(\frac{\pi}{2}(x+1)\right) \sin\left(\frac{\pi}{2}(y+1)\right)$$

对上述 $f(x, y)$ 计算数值解，并与解析解 $w(x, y)$ 比较。

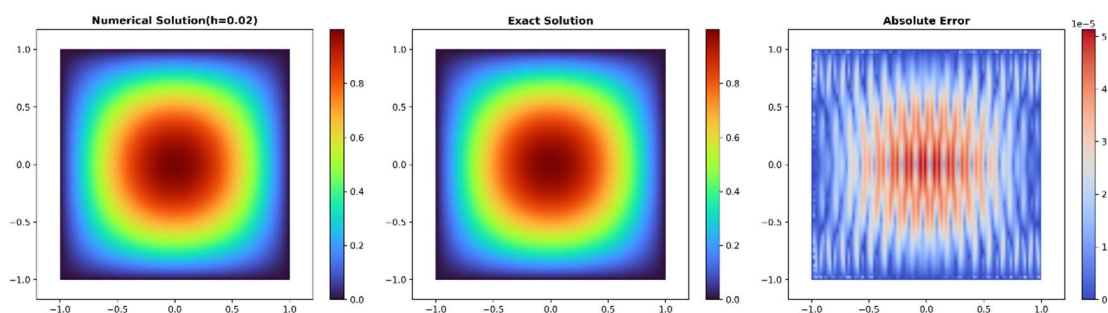


图 2-6 数值解、解析解及绝对误差对比图

选取最大网格尺寸 $h = 0.01$ ，共生成节点46677个，三角形单元92552个。数值解、解析解、绝对误差如图所示。由图 2-6 可见，最大误差在 origin 附近产生，且小于0.00005，相对误差小于0.01%。这说明计算方法是整体稳定的。

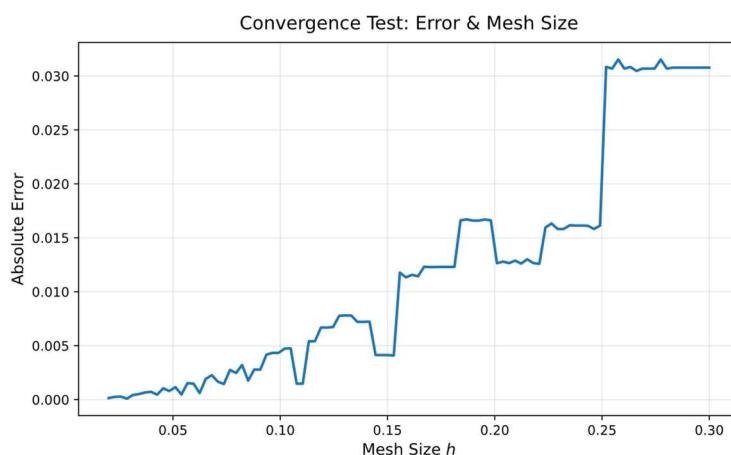


图 2-7 绝对误差随网格尺寸变化图

图 2-7 展示了原点处数值解的绝对误差随网格尺寸的变化，误差随网格尺寸减小而收敛。

2.5 传送门方法的实际应用

传送门提供了一种简单的将高维度二维曲面展开为平面的曲面切割手段。当原本曲面高斯曲率处处为0时，这种展开为保距离变换，能保证方程形式不变。

2.5.1 圆柱面稳态场模拟

考虑长直金属管表面热稳态分布，截取足够长的片段，两端温度接近环境温度，满足 Dirichlet 边界条件。直金属管表面高斯曲率处处为0，沿任意一条母线切

割，可将其展开为平面矩形，保证表面满足的微分方程形式不变。在切割后矩形两侧添加传送门，等效相互连接，与原本圆柱面热稳态分布完全相同。

如图 2-8 所示，其表示了金属管受单一热源影响下，表面热力分布情况。选取求解域为 $[-5,5]^2$ ，传送门长度为10，间距为8。对应的原金属管长为10，周长为8。图中左右两侧传送门对应原金属管的一条母线。

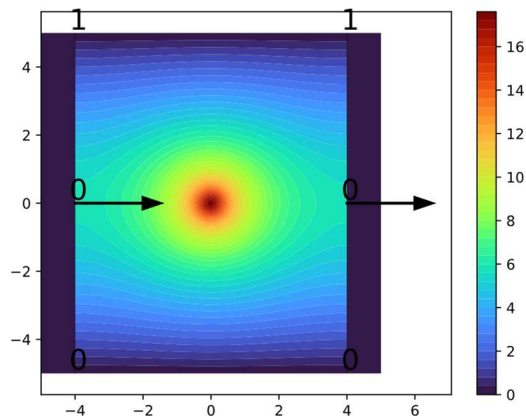


图 2-8 长直金属管表面热稳态分布求解结果

2.5.2 平直环面及平直克莱因瓶表面稳态场模拟

使用两对传送门闭合，可将矩形映射为四维空间中的平直环面 T^2 。对于闭合曲面 Ω ，泊松方程解存在的必要条件是

$$\int_{\Omega} f \, d\Omega = 0$$

可设置一对形状相同，强度相反的场源，使得上式成立，计算结果如图 2-9 (a) 所示。图中传送门近似闭合，以取得良好的数值稳定性。如果将其中一对传送门单门翻转，则矩形对应到了四维空间中平直的克莱因瓶曲面，计算结果如 2-9 (b) 所示。由对称性，两者计算结果相同。

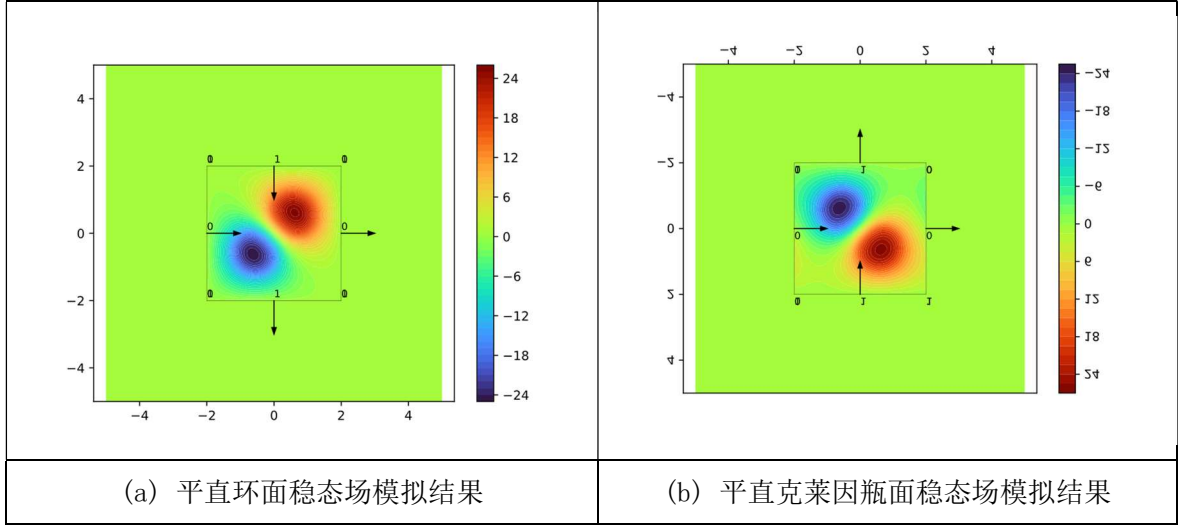


图 2-9 平直环面及平直克莱因瓶表面稳态场模拟

2.5.3 不可展曲面表面场及非泊松方程问题

对球面等不可展曲面，可通过保角变换映射到平面，但该变换一定不是保距变换。非保距变换下，微分方程的形式会发生改变，不再局限于齐次边界的泊松方程，可通过其他有限元方法求解。尽管超出了本文的讨论范畴，但传送门的思路同样适用，进而本文提到的传送门方法仍可以应用于大部分三维及高维空间中二维曲面的有限元求解。

以球面为例，取单位球面经纬曲纹坐标 (u, v) ， $u \in [0, \pi]$ 表示纬度， $v \in [0, 2\pi]$ 表示经度，球面上一点可写作

$$r(u, v) = (\sin u \cos v, \sin u \sin v, \cos u)$$

球面曲面散度可使用拉普拉斯-贝尔特拉米算子计算，泊松方程表示为

$$-\Delta_{LB} w(u, v) = f(u, v)$$

其中

$$\Delta_{LB} w = \frac{1}{\sqrt{EG - F^2}} \left[\partial_u \left(\frac{G}{\sqrt{EG - F^2}} \partial_u w \right) + \partial_v \left(\frac{E}{\sqrt{EG - F^2}} \partial_v w \right) \right]$$

在单位球面上， $E = 1$ ， $F = 0$ ， $G = \sin^2 u$ ，于是

$$\Delta_{LB} w = \frac{1}{\sin u} \left[\frac{\partial}{\partial u} \left(\sin u \frac{\partial w}{\partial u} \right) + \frac{\partial}{\partial v} \left(\frac{1}{\sin u} \frac{\partial w}{\partial v} \right) \right]$$

使用球面墨卡托(G. Mercator)投影，将曲面曲纹坐标 (u, v) 映射为平面直角矩形坐标 (ξ, η) ，其中

$$\xi = v, \quad \eta = \ln \tan \frac{u}{2}$$

于是有

$$dv = d\xi, \quad du = \sin u \, d\eta$$

$$\frac{\partial}{\partial v} = \frac{\partial}{\partial \xi}, \quad \frac{\partial}{\partial u} = \frac{1}{\sin u} \frac{\partial}{\partial \eta}$$

带入原方程整理得到

$$-\left(\frac{\partial^2 \tilde{w}}{\partial \xi^2} + \frac{\partial^2 \tilde{w}}{\partial \eta^2}\right) = \frac{2e^\eta}{1+e^{2\eta}} \cdot \tilde{f}(\xi, \eta)$$

变换后方程仍为泊松方程，可使用本文提到的方法求解，截取纬度范围

$\left[\frac{\pi}{12}, \frac{11\pi}{12}\right]$ ，施加 Dirichlet 边界条件。选取沿赤道分布的高温热源，即

$$f(u, v) = \begin{cases} 100, & |u - \pi/2| < \frac{\pi}{12} \\ 0, & \text{其他情形} \end{cases}$$

于是有

$$\frac{2e^\eta}{1+e^{2\eta}} \tilde{f}(\xi, \eta) = \begin{cases} 100 \frac{2e^\eta}{1+e^{2\eta}}, & \ln \tan \frac{5\pi}{24} < \eta < \ln \tan \frac{7\pi}{24} \\ 0, & \text{其他情形} \end{cases}$$

计算结果如图 2-10 所示

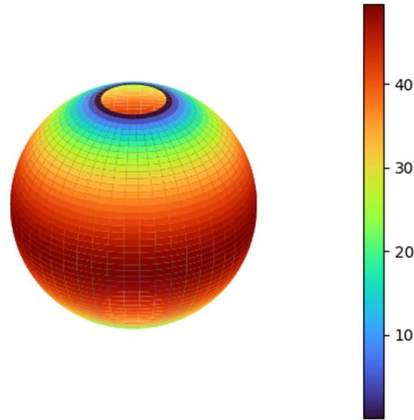


图 2-10 球面热稳态模拟计算结果

3 结论

本文探讨了传送门作用下二维泊松方程的有限元求解方法。通过直接重构网格节点连接关系，在不引入非齐次边界条件的前提下，实现了平面域对圆柱面、环面等复杂拓扑曲面的等效模拟。数值算例表明，该方法具有良好的收敛性与稳定性，能够准确捕捉跨传送门区域的场分布特征。

本方法的核心优势在于：一是避免了三维曲面离散化带来的几何拟合误差，保证了可展曲面问题的精确建模；二是利用二维单元替代三维单元进行计算，显著降低了计算复杂度与资源消耗。同时，本文进一步证明了该方法结合保角变换后在不可展曲面（如球面）问题中的扩展潜力，为处理非平凡拓扑流形上的偏微分方程数值解提供了高效可靠的解决方案。

参 考 文 献

- [1] 余德浩, 汤华中. 微分方程数值解法 (第二版). 北京: 科学出版社, 2018: 318, 319-328.
- [2] Brenner S C, Scott L R. The Mathematical Theory of Finite Element Methods. New York: Springer, 2008: 1-35.
- [3] Geuzaine C, Remacle J F. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. International Journal for Numerical Methods in Engineering, 2009, 79(11).
- [4] Nehari Z. Conformal Mapping. New York: Dover Publications, 1975.

附录（Python 程序）

Python 版本：3.13

第三方库：gmsh, numpy, scipy, matplotlib

代码 1：求解算法定义

portalsolver.py

```
import gmsh
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.tri as mtri
from scipy.sparse import lil_matrix
from scipy.sparse.linalg import cg
import os

class PortalPoissonSolver:
    def __init__(
        self,
        portals,
        domain_side_length: float | tuple = 2.0,
        mesh_size_regular=0.05,
        mesh_size_min=0.005,
        switch_portal=True,
        show_progress=True,
    ):
        # portals: 所有传送门及其位置
        # [[[1x 坐标, 1y 坐标, 1 长度, 1 正方向), (2x 坐标, 2y 坐标, 2 长度, 2 正方向), 是否翻转], ...]
        self.portals = portals
        self.domain_side_length = domain_side_length
        self.mesh_size_regular = mesh_size_regular
        self.mesh_size_min = mesh_size_min
        self.switch_portal = switch_portal # 是否使用传送门
        self.show_progress = show_progress # 是否输出求解进度
        self.portal_points = [(np.array([]), np.array([])) for _ in portals]
        # switched portal element points: 传送门节点变换记录
        # [{(面元 -> (顶点, 初始节点, 目标节点), ...}, ...)]
        self.switched_portal_element_points = [{} for _ in portals]
        self.nodes: np.ndarray
        self.boundary_nodes: np.ndarray
        self.elements: np.ndarray
        self.elements_unswitched: np.ndarray
        self.solution: np.ndarray

    def _calculate_portal_local(self, portal): # 传送门端点位置
        line1, line2, portal_reverse = portal
        x1, y1, l1, theta1 = line1
        x2, y2, l2, theta2 = line2
        alpha1 = theta1 + np.pi / 2
        alpha2 = theta2 + np.pi / 2
        x11, y11, x12, y12 = (
            x1 - l1 / 2 * np.cos(alpha1),
            y1 - l1 / 2 * np.sin(alpha1),
            x1 + l1 / 2 * np.cos(alpha1),
            y1 + l1 / 2 * np.sin(alpha1),
        )
```

```

        sign = -1 if portal_reverse else 1
        x21, y21, x22, y22 = (
            xm2 - sign * l2 / 2 * np.cos(alpha2),
            ym2 - sign * l2 / 2 * np.sin(alpha2),
            xm2 + sign * l2 / 2 * np.cos(alpha2),
            ym2 + sign * l2 / 2 * np.sin(alpha2),
        )

        return x11, y11, x12, y12, x21, y21, x22, y22

    def _distribution_func(self, portal): # 传送门上节点分布
        line1, line2, _ = portal
        _, _, l1, _ = line1
        _, _, l2, _ = line2
        x11, y11, x12, y12, x21, y21, x22, y22 =
self._calculate_portal_local(portal)
        dx1, dy1 = [(x12 - x11) / l1, (y12 - y11) / l1]
        dx2, dy2 = [(x22 - x21) / l2, (y22 - y21) / l2]

        l_argv = (l1 + l2) / 2

        half_distance = l_argv / 2

        size_min = min(l_argv / 50, self.mesh_size_min)
        size_max = min(l_argv / 5, self.mesh_size_regular)

    def _base_distribution_func(x):
        return np.sin(np.pi / 2 * x) ** 2

    def _total_segment_length(size_min, size_max, num_segs):
        d = 0
        for i in range(num_segs):
            d += (
                _base_distribution_func(i / num_segs) * (size_max - size_min)
                + size_min
            )
        return d

    num_segs_min = int(half_distance / size_max) - 1
    num_segs_max = int(half_distance / size_min) + 1
    while True:
        num_segs = (num_segs_min + num_segs_max) // 2
        d = _total_segment_length(size_min, size_max, num_segs)
        if (
            _total_segment_length(size_min, size_max, num_segs - 1)
            <= half_distance
            <= d
        ):
            break
        elif (
            _total_segment_length(size_min, size_max, num_segs + 1)
            >= half_distance
            >= d
        ):
            num_segs += 1
            break
        elif d > half_distance:
            num_segs_max = num_segs
        else:
            num_segs_min = num_segs

```

```

interp_nodes = [0.0] * (2 * num_segs + 1)
for i in range(num_segs - 1):
    interp_nodes[i + 1] = (
        interp_nodes[i]
        + _base_distribution_func(i / num_segs) * (size_max - size_min)
        + size_min
    )
interp_nodes[num_segs] = half_distance
for i in range(num_segs + 1, 2 * num_segs + 1):
    interp_nodes[i] = 2 * half_distance - interp_nodes[2 * num_segs - i]

p1 = [(0, 0)] * len(interp_nodes)
p2 = [(0, 0)] * len(interp_nodes)
for i, d in enumerate(interp_nodes):
    d /= half_distance * 2
    d1 = d * l1
    d2 = d * l2
    p1[i] = (x11 + dx1 * d1, y11 + dy1 * d1)
    p2[i] = (x21 + dx2 * d2, y21 + dy2 * d2)

return p1, p2

def generate_domain_mesh(self, domain_shape="rectangle"):
    if self.mesh_size_regular < self.mesh_size_min:
        raise ValueError("网格尺寸不合理")
    if self.show_progress:
        print("生成网格...")

    def _add_points_to_mesh(points):
        n = len(points)
        point_tags = [0] * n
        line_tags = [0] * (n - 1)
        for i, (x, y) in enumerate(points):
            point_tags[i] = gmsh.model.occ.addPoint(x, y, 0,
self.mesh_size_regular)
        for i in range(n - 1):
            line_tags[i] = gmsh.model.occ.addLine(point_tags[i], point_tags[i
+ 1])
        return point_tags, line_tags

    def _add_portal_to_mesh(portal):
        p1, p2 = self._distribution_func(portal)
        p1_tags, l1_tags = _add_points_to_mesh(p1)
        p2_tags, l2_tags = _add_points_to_mesh(p2)
        return p1_tags, p2_tags, l1_tags + l2_tags

    gmsh.initialize()
    gmsh.option.setNumber("General.Verbosity", 1)
    gmsh.model.add("main_mesh")
    D = self.domain_side_length

    if domain_shape == "rectangle":
        if isinstance(self.domain_side_length, (int, float)):
            A = B = self.domain_side_length
        else:
            A, B = self.domain_side_length
        domain = gmsh.model.occ.addRectangle(-A, -B, 0, 2 * A, 2 * B)
        domain_dimtag = (2, domain)
    elif domain_shape == "circle":
        D = self.domain_side_length

```



```

        circle = gmsh.model.occ.addCircle(0, 0, 0, D)
        loop = gmsh.model.occ.addCurveLoop([circle])
        domain = gmsh.model.occ.addPlaneSurface([loop])
        domain_dimtag = (2, domain)
    else:
        raise ValueError("定义域形状错误")

    portal_points_tags = [] # [(p1_pointTags, p2_pointTags), ...]
    line_dimtags = [] # [(1, lineTag), ...]

    for portal in self.portals:
        p1_tags, p2_tags, l_tags = _add_portal_to_mesh(portal)
        portal_points_tags.append((p1_tags, p2_tags))
        line_dimtags.extend((1, lt) for lt in l_tags)

    outDimTags = [domain_dimtag]
    outMap = None
    try:
        if line_dimtags:
            outDimTags, outMap = gmsh.model.occ.fragment(
                [domain_dimtag], line_dimtags
            )
        gmsh.model.occ.synchronize()

        if line_dimtags:
            for dim, lt in line_dimtags:
                gmsh.model.mesh.setTransfiniteCurve(lt, 1)
    except Exception as e:
        print(e)
        raise ValueError("网格生成错误, 请检查传送门是否交叉")

    gmsh.option.setNumber("Mesh.CharacteristicLengthMax",
self.mesh_size_regular)
    gmsh.model.mesh.generate(2)

    node_tags, node_coords, _ = gmsh.model.mesh.getNodes()
    self.nodes = np.array(node_coords).reshape(-1, 3)[:,:2]
    tag2idx = {int(t): i for i, t in enumerate(node_tags)}

    solve_surfs = [dt for dt in outDimTags if dt[0] == 2]

    all_tris = []
    for dim, stag in solve_surfs:
        etypes, etags, enodes = gmsh.model.mesh.getElements(dim, stag)
        if not enodes:
            continue
        tris = np.array(enodes[0]).reshape(-1, 3)
        all_tris.append(tris)

    self.elements_unswitched = np.vectorize(tag2idx.get)(np.vstack(all_tris))
    self.elements = self.elements_unswitched.copy()

    self.portal_points = [(np.array([]), np.array([])) for _ in self.portals]

    for i, portal in enumerate(self.portals):
        p1_occ_pts, p2_occ_pts = portal_points_tags[i]
        # p1_occ_pts = p1_occ_pts[1:-1]
        # p2_occ_pts = p2_occ_pts[1:-1]
        try:
            p1_idx, p2_idx = [], []

```

```

        for pt in p1_occ_pts:
            ntags, _, _ = gmsh.model.mesh.getNodes(0, pt)
            if len(ntags) > 0:
                p1_idx.append(tag2idx[int(ntags[0])])
        for pt in p2_occ_pts:
            ntags, _, _ = gmsh.model.mesh.getNodes(0, pt)
            if len(ntags) > 0:
                p2_idx.append(tag2idx[int(ntags[0])])
    except Exception as e:
        print(e)
        raise ValueError("网格生成错误, 请检查传送门是否重叠")

    self.portal_points[i] = (np.array(p1_idx), np.array(p2_idx))

def _tri_centroid_xy(a, b, c):
    x1, y1 = self.nodes[a]
    x2, y2 = self.nodes[b]
    x3, y3 = self.nodes[c]
    return (x1 + x2 + x3) / 3, (y1 + y2 + y3) / 3

self.switched_portal_element_points = [{ } for _ in self.portals]
if self.switch_portal:
    for i, (p1, p2) in enumerate(self.portal_points):
        line1, line2, _ = self.portals[i]
        x, y, _, theta = zip(line1, line2)

        inner_nodes = {int(n): 0 for n in p1} | {int(n): 1 for n in p2}
        inner_nodes_map = {int(a): int(b) for a, b in zip(p1, p2)} | {
            int(b): int(a) for a, b in zip(p1, p2)
        }

        switch_rule = set()
        for j, Ele in enumerate(self.elements):
            for k, node_idx in enumerate(Ele):
                node_idx = int(node_idx)
                if node_idx in inner_nodes:
                    p_id = inner_nodes[node_idx]
                    a, b, c = map(int, Ele)

                    tri_x, tri_y = _tri_centroid_xy(a, b, c)
                    co, si = np.cos(theta[p_id]), np.sin(theta[p_id])
                    dx, dy = tri_x - x[p_id], tri_y - y[p_id]
                    if co * dx + si * dy < 0:
                        st, ed = node_idx, inner_nodes_map[node_idx]
                        switch_rule.add((j, k, st, ed))

        for j, k, st, ed in switch_rule:
            self.elements[j][k] = ed
            self.switched_portal_element_points[i].setdefault(j,
set()).add(
                (k, st, ed)
            )

    boundary_curves = gmsh.model.getBoundary(
        solve_surfs, oriented=False, recursive=False
    )
    boundary_points = gmsh.model.getBoundary(
        solve_surfs, oriented=False, recursive=True
    )

```

```

        bnd_node_tags = set()
        for dim, tag in boundary_curves + boundary_points:
            ntags, _, _ = gmsh.model.mesh.getNodes(dim, tag)
            bnd_node_tags.update(map(int, ntags))

        self.boundary_nodes = np.array([tag2idx[t] for t in bnd_node_tags],
dtype=int)

        if self.show_progress:
            print(f"节点数量: {len(self.nodes)}, 面元数量: {len(self.elements)}")
            gmsh.finalize()

    def plot_portal(self):
        # fmt: off
        for portal_idx, portal in enumerate(self.portals):
            line1, line2, reverse = portal
            x11, y11, l1, theta1 = line1
            x12, y12, l2, theta2 = line2
            x21, y21, x22, y22 =
self._calculate_portal_local(portal)

            plt.plot([x11, x12], [y11, y12], color="black", linewidth=0.2)
            plt.plot([x21, x22], [y21, y22], color="black", linewidth=0.2)

            l_argv = (l1 + l2) / 2

            arrow_len = l_argv / 5
            linewidth = arrow_len
            head_width = arrow_len / 6
            head_length = arrow_len / 4

            fontsize = l_argv * 2
            dx1, dy1 = arrow_len * np.cos(theta1), arrow_len * np.sin(theta1)
            dx2, dy2 = arrow_len * np.cos(theta2), arrow_len * np.sin(theta2)
            plt.arrow(x11, y11, dx1, dy1, head_width=head_width,
head_length=head_length,
                    fc='black', ec='black', linewidth=linewidth)
            plt.arrow(x12, y12, dx2, dy2, head_width=head_width,
head_length=head_length,
                    fc='black', ec='black', linewidth=linewidth)

            offset = 0.1
            plt.text(x11+offset, y11+offset, f'{portal_idx}',
                    fontsize=fontsize, color='black', ha='center')
            plt.text(x12+offset, y12+offset, f'{portal_idx}',
                    fontsize=fontsize, color='black', ha='center')

            plt.text(x21+offset, y21+offset, '0', fontsize=fontsize,
color='black', ha='center')
            plt.text(x22+offset, y22+offset, '1', fontsize=fontsize,
color='black', ha='center')
            # fmt: on

    def plot_mesh(
        self,
        show_ele_tags=False,

```

```

        plot_boundary_nodes=True,
        plot_portal_points=True,
        plot_portal=True,
        fig_name="meshfig.svg",
    ): # 绘制网格
        if not hasattr(self, "elements"):
            self.generate_domain_mesh()
        if self.show_progress:
            print("绘制网格图像...")
        D = self.domain_side_length

        for i, (a, b, c) in enumerate(self.elements):
            x1, y1 = self.nodes[a]
            x2, y2 = self.nodes[b]
            x3, y3 = self.nodes[c]
            x = (x1 + x2 + x3) / 3
            y = (y1 + y2 + y3) / 3

            d1 = np.sqrt((x3 - x2) ** 2 + (y3 - y2) ** 2)
            d2 = np.sqrt((x1 - x3) ** 2 + (y1 - y3) ** 2)
            d3 = np.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

            min_d = min(d1, d2, d3)

            linewidth = min_d
            plt.plot(
                [x1, x2, x3, x1],
                [y1, y2, y3, y1],
                color="r",
                linewidth=linewidth,
                zorder=0,
            )
            fontsize = min_d**2 * 5
            if show_ele_tags:
                plt.text(x, y, f"{i}", fontsize=fontsize, ha="center",
va="center")

        if plot_boundary_nodes:
            point_size = self.mesh_size_regular * 4
            plt.scatter(
                self.nodes[self.boundary_nodes, 0],
                self.nodes[self.boundary_nodes, 1],
                color="b",
                s=point_size,
            )

        if plot_portal_points:
            xs, ys = [], []
            for pls, p2s in self.portal_points:
                for t1, t2 in zip(pls, p2s):
                    for t in (t1, t2):
                        xs.append(self.nodes[t][0])
                        ys.append(self.nodes[t][1])
            plt.plot(xs, ys, "ko", markersize=0.3)

        if plot_portal:
            self.plot_portal()

        plt.axis("equal")
        os.makedirs("./output", exist_ok=True)

```

```

plt.savefig(os.path.join("./output", fig_name))
plt.show()

def solve_poisson(self, func_domain, solve_method="cg"): # 求解方程
# Galerkin 法
if not hasattr(self, "elements"):
    self.generate_domain_mesh()
if self.show_progress:
    print("求解方程...")

def _fill_KF(self, K, F):
    for idx_ele in range(len(self.elements)):
        id1, id2, id3 = self.elements_unswitched[idx_ele]
        x1, y1 = self.nodes[id1]
        x2, y2 = self.nodes[id2]
        x3, y3 = self.nodes[id3]
        xc = (x1 + x2 + x3) / 3
        yc = (y1 + y2 + y3) / 3
        b = np.array([y2 - y3, y3 - y1, y1 - y2])
        c = np.array([x3 - x2, x1 - x3, x2 - x1])
        area = 0.5 * abs((x2 - x1) * (y3 - y1) - (y2 - y1) * (x3 - x1))

        A_loc = np.zeros((3, 3))
        for i in range(3):
            for j in range(3):
                A_loc[i, j] = (b[i] * b[j] + c[i] * c[j]) / (4 * area)

        f_val = func_domain(xc, yc)
        B_loc = area * f_val / 3 * np.ones(3)
        ids = self.elements[idx_ele]

        for i in range(3):
            F[ids[i]] += B_loc[i]
            for j in range(3):
                K[ids[i], ids[j]] += A_loc[i, j]

    for n in self.boundary_nodes:
        K[n, :] = 0
        K[:, n] = 0
        K[n, n] = 1
        F[n] = 0
    return

num_N = len(self.nodes)
F = np.zeros(num_N)
if solve_method == "direct":
    K = np.zeros((num_N, num_N))
    _fill_KF(self, K, F)
    U = np.linalg.solve(K, F)
elif solve_method == "cg":
    K = lil_matrix((num_N, num_N))
    _fill_KF(self, K, F)
    K_csr = K.tocsr()
    U, _ = cg(K_csr, F, rtol=1e-6, maxiter=5000)
else:
    raise ValueError("线性求解器名称错误")

self.solution = U
return U

```

```

def evaluate(self, x, y): # 计算 x, y 处解的函数值
    if not hasattr(self, "solution"):
        print("尚未求解, 请先调用 solve_poisson")
    if not hasattr(self, "_triang_unswitched"):
        xs, ys = self.nodes[:, 0], self.nodes[:, 1]
        elements_unswitched = self.elements_unswitched
        self._triang_unswitched = mtri.Triangulation(xs, ys,
elements_unswitched)
        self._trifinder_unswitched = self._triang_unswitched.get_trifinder()

    tri_id: int = self._trifinder_unswitched(x, y) # type: ignore
    if tri_id == -1:
        return np.nan
    a, b, c = self.elements_unswitched[tri_id]
    x1, y1 = self.nodes[a]
    x2, y2 = self.nodes[b]
    x3, y3 = self.nodes[c]

    va, vb, vc = self.elements[tri_id]
    u1 = self.solution[va]
    u2 = self.solution[vb]
    u3 = self.solution[vc]

    denom = (y2 - y3) * (x1 - x3) + (x3 - x2) * (y1 - y3)

    w1 = ((y2 - y3) * (x - x3) + (x3 - x2) * (y - y3)) / denom
    w2 = ((y3 - y1) * (x - x3) + (x1 - x3) * (y - y3)) / denom
    w3 = 1.0 - w1 - w2

    return w1 * u1 + w2 * u2 + w3 * u3

def plot_solution(
    self,
    source_boundary=[],
    draw_contours=True,
    plot_portal=True,
    fig_name="solutionfig.svg",
): # 绘制解图像
    if not hasattr(self, "solution"):
        print("尚未求解, 请先调用 solve_poisson")
    if self.show_progress:
        print("绘制解图像...")
    x = list(self.nodes[:, 0].copy())
    y = list(self.nodes[:, 1].copy())
    elements = list(self.elements_unswitched.copy())
    solution = list(self.solution.copy())
    switched_all_portals = self.switched_portal_element_points
    for switched_element in switched_all_portals:
        for j in switched_element:
            for k, st, ed in switched_element[j]:
                elements[j][k] = len(x)
                x.append(x[st])
                y.append(y[st])
                solution.append(solution[ed])

    triang = mtri.Triangulation(x, y, elements)
    CMAP = "turbo"
    if draw_contours:
        plt.tricontourf(triang, solution, levels=50, cmap=CMAP)
    else:

```

```

        plt.tripcolor(triang, solution, cmap=CMAP)
    plt.colorbar()

    if source_boundary:
        for boundary in source_boundary:
            plt.plot(
                boundary[0],
                boundary[1],
                color="black",
                linewidth=self.mesh_size_min,
            )

    if plot_portal:
        self.plot_portal()

    plt.axis("equal")
    os.makedirs("./output", exist_ok=True)
    plt.savefig(os.path.join("./output", fig_name))
    plt.show()

if __name__ == "__main__":
    portals = [
        [(1, 0, 2, 0), (-1, 0, 2, np.pi / 3), False],
        [(0, 2, 2, np.pi / 2), (0, -2, 2, np.pi / 3), False],
    ]
    solver = PortalPoissonSolver(
        portals,
        domain_side_length=5,
        mesh_size_regular=0.6,
        mesh_size_min=0.3,
        switch_portal=True,
        show_progress=True,
    )
    solver.generate_domain_mesh(domain_shape="circle") # "rectangle", "circle"

    # 绘制网格图像，使用时网格尺寸不宜过小，否则绘图耗时将会极长
    solver.plot_mesh(
        show_ele_tags=True,
        plot_boundary_nodes=True,
        plot_portal_points=True,
        plot_portal=True,
        fig_name="meshfig.svg",
    )

    xc, yc = 0, 0
    R = 0.3
    G = 100

    def source_func(x, y):
        dx, dy = x - xc, y - yc
        r = np.sqrt((dx) ** 2 + (dy) ** 2)
        # r = abs(dx)+abs(dy)
        # r = max(abs(dx), abs(dy))
        if r < R:
            return G
        else:
            return 0

    source_boundary = [] # 仅用于绘图，非必须参数

```

```

t = np.linspace(0, 2 * np.pi, 100)
xs, ys = xc + R * np.cos(t), yc + R * np.sin(t)
source_boundary.append((xs, ys))

solver.solve_poisson(source_func, solve_method="cg")
solver.plot_solution(
    source_boundary=source_boundary, draw_contours=True, plot_portal=True
)

print(solver.evaluate(1, 1)) # 计算特定点函数值

```

代码 2： 求解结果展示

solve_express.py

```

from portalsolver import PortalPoissonSolver, np
import matplotlib.pyplot as plt
from matplotlib.colors import Normalize
from matplotlib.cm import ScalarMappable
from matplotlib import colormaps
import os

# fmt:off
class CaseRun(PortalPoissonSolver):
    def __init__(
        self,
        portals, domain_shape, domain_size, mesh_size_regular, mesh_size_min,
        source_func, source_boundary, fig_name,
        switch_portal = True,
        show_progress = True,
        solve_method = "cg"
    ):
        super().__init__(
            portals=portals,
            domain_side_length=domain_size,
            mesh_size_regular=mesh_size_regular,
            mesh_size_min=mesh_size_min,
            switch_portal=switch_portal,
            show_progress=show_progress
        )

        self.generate_domain_mesh(domain_shape=domain_shape)
        self.solve_poisson(source_func, solve_method=solve_method)
        self.plot_solution(
            source_boundary=source_boundary,
            draw_contours=True,
            plot_portal=True,
            fig_name=fig_name
        )

def plot_sphere(f, fig_name="sphere.svg"):
    import numpy as np
    import matplotlib.pyplot as plt
    from matplotlib import cm

    err = 1e-6

```

```

u = np.linspace(np.pi/12+err, 11*np.pi/12-err, 200)
v = np.linspace(-np.pi+err, np.pi-err, 300)
U, V = np.meshgrid(u, v, indexing='ij')

X = np.sin(U) * np.cos(V)
Y = np.sin(U) * np.sin(V)
Z = np.cos(U)

F = np.empty(U.shape)
for i in range(U.shape[0]):
    for j in range(U.shape[1]):
        if str(f(U[i, j], V[i, j])) == "nan":
            print(U[i, j], V[i, j])
        F[i, j] = f(U[i, j], V[i, j])
cmap = colormaps["turbo"]
norm = Normalize(vmin=np.nanmin(F), vmax=np.nanmax(F))
C = cmap(norm(F))

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, facecolors=C, linewidth=0, shade=False)
ax.set_xlim(-1, 1)
ax.set_ylim(-1, 1)
ax.set_zlim(-1, 1)
ax.set_box_aspect((1,1,1))
ax.axis('off')
mappable = ScalarMappable(norm=norm, cmap=cmap)
mappable.set_array([])

fig.colorbar(mappable, ax=ax)

os.makedirs("./output", exist_ok=True)
plt.savefig(os.path.join("./output", fig_name))
plt.show()

p1 = [[(1, 0, 2, 0), (-1, 0, 2, 0), False]]
p2 = [[(1, 0, 2, 0), (-1, 0, 2, np.pi), False]]
p3 = [
    [(1, 0, 2, 0), (-1, 0, 2, np.pi / 3), False],
    [(0, 2, 2, np.pi / 2), (0, -2, 2, np.pi / 3), False],
]
p4 = [[(1, 1, 3, np.pi / 2), (-1, -1, 3, -np.pi / 2), False]]
p5 = [
    [(1, 2, 2, 0), (-0.5, 1, 2, np.pi/6), True],
    [(1, -2, 2, np.pi/2), (-0.5, -1, 2, 0), True],
    [(0.4, 0, 2, 0), (4, 0, 2, 0), False],
]
p6 = [[(-4, 0, 9.99, 0), (4, 0, 9.99, 0), False]]
p7 = [
    [(-2, 0, 4, 0), (2, 0, 4, 0), True],
    [(0, 2.001, 4, -np.pi / 2), (0, -2.001, 4, -np.pi / 2), True],
]
p8 = [
    [(-2, 0, 4, 0), (2, 0, 4, 0), False],
    [(0, 2.001, 4, -np.pi / 2), (0, -2.001, 4, -np.pi / 2), True],
]
p9 = [[(-np.pi, 0, 4, 0), (np.pi, 0, 4, 0), False]]

```

```

def s_cir(x, y):
    G = 100
    R = 0.3
    dx, dy = x - 0, y - 0
    r = np.sqrt((dx) ** 2 + (dy) ** 2)
    res = 0
    if r < R:
        res = G
    return res

def s_rtg(x, y):
    C, H, L, D, G = 0, 6, 18, 5, 100
    res = 0
    xl, xr = C - L / 2, C + L / 2
    if xl <= x <= xr:
        if H - D <= y <= H:
            res = G
        if -H <= y <= -H + D:
            res = -G
    return res

def s_two_cir(x, y):
    G = 100
    R = 0.7
    dx, dy = x - 0.5, y - 0.5
    r = np.sqrt((dx) ** 2 + (dy) ** 2)
    res = 0
    if r < R:
        res = G
    dx, dy = x + 0.5, y + 0.5
    r = np.sqrt((dx) ** 2 + (dy) ** 2)
    if r < R:
        res = -G
    return res

H_sphere_xi = -np.log(np.tan(np.pi / 24))
def s_sphere_rtg(xi, eta):
    eta_min = np.log(np.tan(5 * np.pi / 24))
    eta_max = np.log(np.tan(7 * np.pi / 24))

    res = 0
    if eta_min < eta < eta_max:
        res = 100
    exp_eta = np.exp(eta)
    res *= 2 * exp_eta / (1 + exp_eta**2)

    return res

CaseRun(p1, "circle", 5, 0.1, 0.05, s_cir, [], "圆形边界, 中心源, 平行传送门.svg")
CaseRun(p2, "circle", 5, 0.1, 0.05, s_cir, [], "圆形边界, 中心源, 平行传送门, 等效无门.svg")
CaseRun(p3, "circle", 5, 0.1, 0.05, s_cir, [], "圆形边界, 中心源, 2 对门.svg")
CaseRun(p4, "rectangle", 10, 0.2, 0.1, s_rtg, [], "方形边界, 平行条形场源, 1 对门.svg")
CaseRun(p5, "circle", 5, 0.1, 0.05, s_cir, [], "圆形边界, 中心源, 3 对门.svg")
CaseRun(p6, "rectangle", 5, 0.1, 0.05, s_cir, [], "圆柱面热稳态模拟.svg")
CaseRun(p7, "rectangle", 5, 0.1, 0.05, s_two_cir, [], "环面稳态场模拟.svg")
CaseRun(p8, "rectangle", 5, 0.1, 0.05, s_two_cir, [], "克莱因瓶表面稳态场模

```

```

拟.svg")
case9 = CaseRun(p9, "rectangle", (np.pi+0.001,H_sphere_xi), 0.1, 0.05,
s_sphere_rtg, [], "球面热稳态模拟.svg")
plot_sphere(lambda x, y: case9.evaluate(y, np.log(np.tan(x/2))), fig_name="球面热
稳态还原")

```

代码 3： 绘制网格示意图

mesh_express.py

```

from portalsolver import PortalPoissonSolver, np

portals = [
    [(3, 0, 5, 0), (-1, 0, 5, np.pi / 3), False],
]

def prime_mesh(): # 初始网格示意图
    solver = PortalPoissonSolver(
        [],
        domain_side_length=5,
        mesh_size_regular=3,
        mesh_size_min=3,
        switch_portal=False,
        show_progress=False,
    )

    solver.generate_domain_mesh(domain_shape="circle")

    solver.plot_mesh(
        show_ele_tags=True,
        plot_boundary_nodes=True,
        plot_portal_points=True,
        plot_portal=True,
        fig_name="初始网格示意图.svg",
    )

def portal_forced_mesh(): # 添加传送门后网格示意图
    solver = PortalPoissonSolver(
        portals,
        domain_side_length=5,
        mesh_size_regular=3,
        mesh_size_min=3,
        switch_portal=False,
        show_progress=False,
    )

    solver.generate_domain_mesh(domain_shape="circle")

    solver.plot_mesh(
        show_ele_tags=True,
        plot_boundary_nodes=True,
        plot_portal_points=True,
        plot_portal=True,
        fig_name="添加传送门后网格示意图.svg",
    )

def switched_mesh(): # 变换后网格示意图
    solver = PortalPoissonSolver(

```

```

        portals,
        domain_side_length=5,
        mesh_size_regular=3,
        mesh_size_min=3,
        switch_portal=True,
        show_progress=False,
    )

    solver.generate_domain_mesh(domain_shape="circle")

    solver.plot_mesh(
        show_ele_tags=True,
        plot_boundary_nodes=True,
        plot_portal_points=True,
        plot_portal=True,
        fig_name="变换后网格示意图.svg",
    )

def square_domain_mesh(): # 方形定义域下的网格
    solver = PortalPoissonSolver(
        portals,
        domain_side_length=5,
        mesh_size_regular=3,
        mesh_size_min=3,
        switch_portal=False,
        show_progress=False,
    )

    solver.generate_domain_mesh(domain_shape="rectangle")

    solver.plot_mesh(
        show_ele_tags=True,
        plot_boundary_nodes=True,
        plot_portal_points=True,
        plot_portal=True,
        fig_name="方形定义域网格.svg",
    )

prime_mesh()
portal_forced_mesh()
switched_mesh()
square_domain_mesh()

```

代码 4： 绘制传送门示意图

portalsolver.py

```

import matplotlib.pyplot as plt
import numpy as np

def plot_portal(portals, fig_name):
    # fmt: off
    for portal_idx, portal in enumerate(portals):
        line1, line2, reverse = portal
        xm1, ym1, l1, theta1 = line1
        xm2, ym2, l2, theta2 = line2

```

```

alpha1 = theta1 + np.pi / 2
alpha2 = theta2 + np.pi / 2
x11, y11, x12, y12 = (
    x11 - l1 / 2 * np.cos(alpha1),
    y11 - l1 / 2 * np.sin(alpha1),
    x11 + l1 / 2 * np.cos(alpha1),
    y11 + l1 / 2 * np.sin(alpha1),
)
sign = -1 if reverse else 1
x21, y21, x22, y22 = (
    x21 - sign * l2 / 2 * np.cos(alpha2),
    y21 - sign * l2 / 2 * np.sin(alpha2),
    x21 + sign * l2 / 2 * np.cos(alpha2),
    y21 + sign * l2 / 2 * np.sin(alpha2),
)

plt.plot([x11, x12], [y11, y12], color="black", linewidth=1)
plt.plot([x21, x22], [y21, y22], color="black", linewidth=1)

l_argv = (l1 + l2) / 2

arrow_len = l_argv / 8
linewidth = arrow_len
head_width = arrow_len / 6
head_length = arrow_len / 6

fontsize = l_argv * 4
dx1, dy1 = arrow_len * np.cos(theta1), arrow_len * np.sin(theta1)
dx2, dy2 = arrow_len * np.cos(theta2), arrow_len * np.sin(theta2)
plt.arrow(x11, y11, dx1, dy1, head_width=head_width,
head_length=head_length,
fc='black', ec='black', linewidth=linewidth)
plt.arrow(x21, y21, dx2, dy2, head_width=head_width,
head_length=head_length,
fc='black', ec='black', linewidth=linewidth)

offset = 0.1
plt.text(x11+offset, y11+offset, '0', fontsize=fontsize, color='black',
ha='center')
plt.text(x12+offset, y12+offset, '1', fontsize=fontsize, color='black',
ha='center')
plt.text(x21+offset, y21+offset, '0', fontsize=fontsize, color='black',
ha='center')
plt.text(x22+offset, y22+offset, '1', fontsize=fontsize, color='black',
ha='center')
plt.axis("equal")
plt.xlim(-3,3)
plt.ylim(-3,3)
plt.savefig("./output/" + fig_name)
plt.show()

p1 = [[(1, 0, 3, 0), (-1, 0, 3, np.pi / 6), False]]
p2 = [[(1, 0, 3, 0), (-1, 0, 3, np.pi / 6), True]]
plot_portal(p1, "传送门示意图, 未翻转.svg")
plot_portal(p2, "传送门示意图, 翻转.svg")

```

代码 5： 稳定性分析

stability_express.py

```
import numpy as np
import matplotlib.pyplot as plt
from portalsolver import PortalPoissonSolver

pi = np.pi
def exact_solution(x, y): # 解析解
    return np.sin(pi*(x+1)/2) * np.sin(pi*(y+1)/2)

def source_func(x, y): # 泊松方程右端源项
    return (pi**2 / 2) * np.sin(pi*(x+1)/2) * np.sin(pi*(y+1)/2)

def plot_error():
    solver = PortalPoissonSolver(
        portals=[],
        domain_side_length=1.0,
        mesh_size_regular=0.01,
        mesh_size_min=0.01,
        switch_portal=True,
        show_progress=True
    )

    solver.generate_domain_mesh(domain_shape="rectangle")
    solver.solve_poisson(func_domain=source_func)

    print("绘制数值解、解析解对比图...")
    N = 100
    x = np.linspace(-1, 1, N)
    y = np.linspace(-1, 1, N)
    X, Y = np.meshgrid(x, y)

    numerical = np.zeros_like(X)
    exact = np.zeros_like(X)
    error = np.zeros_like(X)

    for i in range(N):
        for j in range(N):
            xi = X[i, j]
            yi = Y[i, j]
            numerical[i, j] = solver.evaluate(xi, yi)
            exact[i, j] = exact_solution(xi, yi)
            error[i, j] = np.abs(numerical[i, j] - exact[i, j])

    numerical = np.nan_to_num(numerical, nan=0)
    exact = np.nan_to_num(exact, nan=0)
    error = np.nan_to_num(error, nan=0)

    l2_error = np.sqrt(np.mean(error ** 2))
    max_error = np.max(error)

    print(f"L2 平均误差: {l2_error:.6f}")
    print(f"最大绝对误差: {max_error:.6f}")

    plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    cmap = "turbo"
```

```

    im1 = axes[0].pcolormesh(X, Y, numerical, cmap=cmap, shading='gouraud')
    axes[0].set_title("Numerical Solution(h=0.02)", fontsize=12,
fontweight='bold')
    axes[0].axis("equal")
    plt.colorbar(im1, ax=axes[0])

    im2 = axes[1].pcolormesh(X, Y, exact, cmap=cmap, shading='gouraud')
    axes[1].set_title("Exact Solution", fontsize=12, fontweight='bold')
    axes[1].axis("equal")
    plt.colorbar(im2, ax=axes[1])

    im3 = axes[2].pcolormesh(X, Y, error, cmap="coolwarm", shading='gouraud')
    axes[2].set_title(f"Absolute Error", fontsize=12, fontweight='bold')
    axes[2].axis("equal")
    plt.colorbar(im3, ax=axes[2])

    plt.tight_layout()
    plt.savefig("../output/稳定性分析-数值解与解析解对比.png", dpi=300)
    plt.show()

def calculate_error(mesh_size, x=0, y=0):
    solver = PortalPoissonSolver(
        portals=[],
        domain_side_length=1.0,
        mesh_size_regular=mesh_size,
        mesh_size_min=mesh_size,
        switch_portal=True,
        show_progress=False
    )

    solver.generate_domain_mesh(domain_shape="rectangle")
    solver.solve_poisson(func_domain=source_func)

    numerical = solver.evaluate(x, y)
    exact = exact_solution(x, y)
    error = np.abs(numerical - exact)

    return error

def plot_error_h(hl=0.02, hr=0.3):
    # hs = np.logspace(np.log10(hl), np.log10(hr), 20)
    hs = np.linspace(hl, hr, 100)
    errors = np.zeros_like(hs)

    for i, h in enumerate(hs):
        errors[i] = calculate_error(h)

    plt.figure(figsize=(8, 5))
    plt.plot(hs, errors, linewidth=2)

    plt.xlabel('Mesh Size $h$', fontsize=12)
    plt.ylabel('Absolute Error', fontsize=12)
    plt.title('Convergence Test: Error & Mesh Size', fontsize=14, pad=10)

    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig("../output/收敛性测试-误差-网格尺寸图.svg")
    plt.show()
plot_error()
plot_error_h()

```
